Multi-Vendor E-Commerce Backend

Daily Execution Guide: Day 1 to Day 15

Har din ka kaam, banane wali APIs, request/response aur test cases

FastAPI | PostgreSQL | SQLAlchemy | Pydantic | JWT | Alembic Migrations

Backend Only | COD Only | Swagger & Postman Testing

Contents

Section	Kya hai
1	Is guide ko kaise use karna hai (daily routine, conventions)
2	Project summary, roles aur access matrix
3	Test data (har din ke test isi data par hon ge)
4	Database summary (12 tables)
5	Business rules (checkout, status, COD, reviews)
6	Master API list: kaun si API kis din banegi (total 38 endpoints)
7	Day 1 se Day 15: har din ka complete plan
8	Final checklist aur end-to-end demo

1. Is Guide ko Kaise Use Karna Hai

Roz ka routine

1. **Subah:** us din ka section parhein: Goal, Steps aur APIs table.
2. **Kaam:** Steps ko order mein karein. Har API banane ke baad foran Swagger (/docs) mein test karein.
3. **Test:** Day ke Test Cases table ke har row ko Swagger ya Postman se chalayein aur Expected result se match karein.
4. **Shaam:** End-of-Day checklist tick karein. Agar koi test fail hai to agla din shuru na karein.
5. **Git:** Day ke end par diya hua commit message use kar ke commit karein. Migration ho to wo bhi commit karein.

Rule: Ek din ka kaam complete aur test hone ke baad hi agla din shuru hoga. Database schema sirf Alembic migrations se badlega, **create_all() kabhi use nahi hoga**.

API Conventions (poore project mein same)

Cheez	Rule
Create success	201 Created + naya object
Read / Update / Delete success	200 OK. Delete par {"message": "... deleted"} ya object
Validation error	422 (Pydantic / FastAPI automatically)
Business rule fail	400 (e.g. empty cart, insufficient stock, invalid status change)
Token missing/invalid/expired	401 Unauthorized
Role galat ya doosre ka resource	403 Forbidden
Record nahi mila	404 Not Found
Duplicate (email, review, etc.)	409 Conflict
Error body format	`{"detail": "message"}`
Auth header	`Authorization: Bearer <token>`
Pagination	Query: skip (default 0), limit (default 20, max 100)
Money	Numeric(10,2), response mein string/number 2 decimals
Time	UTC datetime (ISO 8601)

Postman Setup (Day 5 se use hoga)

- Collection banayein: **Ecommerce Backend**. Har Day ke liye ek folder: **Day 5 - Auth**, **Day 7 - Categories**, etc.
- Environment variables: `base_url` = http://localhost:8000, `buyer_token`, `buyer2_token`, `seller_token`, `seller2_token`, `admin_token`.
- Har request ke Authorization tab mein: Bearer Token = {{buyer_token}} (ya jo role chahiye).
- Login request ke Tests tab mein token auto-save: `pm.environment.set('buyer_token', pm.response.json().access_token)`.
- Swagger mein: upar **Authorize** button se token paste karein.

2. Project Summary aur Access Matrix

Multi-vendor marketplace: Sellers products bechte hain, Buyers cart se COD order karte hain, Admin sab control karta hai. Koi frontend nahi; sirf APIs.

Role	Kaise banta hai	Kya kar sakta hai
Buyer	Public signup (role = buyer)	Browse, cart, address, checkout, apne orders, review
Seller	Public signup (role = seller)	Apne products + images, apne seller orders ka status
Admin	Sirf seed script (public signup se nahi)	Users, categories, sab orders, COD confirm, cancellation

Access Matrix

Feature	Public	Buyer	Seller	Admin
Register / Login	Yes	-	-	-
View categories / products / reviews	Yes	Yes	Yes	Yes
Category create/update/delete	No	No	No	Yes
Product create/update/deactivate/images	No	No	Own only	No (monitor only)
Addresses, Cart, Checkout	No	Own only	No	No
View own order (main)	No	Own only	No	All
Seller orders list / status update	No	No	Own only	View all
Payment details	No	Own order	No	All
Confirm COD cash	No	No	No	Yes
Cancel order, manage users	No	No	No	Yes
Post review	No	Verified buyer	No	No

3. Test Data (har din yehi use karein)

Is data ko Day 5 se Day 14 tak tests mein use kiya jayega, taake expected results fix rahein.

Naam	Email	Password	Role
Ali Khan	ali@example.com	Ali@12345	buyer (buyer_token)
Sara Noor	sara@example.com	Sara@12345	buyer 2 (buyer2_token)
Ahmed Khan	ahmed@example.com	Ahmed@12345	seller (seller_token)
Bilal Raza	bilal@example.com	Bilal@12345	seller 2 (seller2_token)
Admin	admin@example.com	Admin@12345	admin (seed script se, admin_token)

Product	Seller	Category	Price	Stock
Samsung Mobile	Ahmed	Mobiles	50000.00	10
Headphones	Ahmed	Accessories	3000.00	20
Nike Shoes	Bilal	Shoes	8000.00	5

Standard checkout scenario: Ali ke cart mein Mobile x1, Headphones x2, Shoes x1. Total = 50000 + 6000 + 8000 = **Rs. 64,000**. Ahmed ka hissa 56,000 aur Bilal ka 8,000.

Test address (Ali): Ali Khan, 03001234567, House 12 Street 5, Lahore, 54000, Pakistan.

4. Database Summary (12 Tables)

Table	Columns	Key constraints
users	id, name, email, password_hash, role, is_active, created_at	email UNIQUE; role in (buyer, seller, admin)
categories	id, name, description, created_at	name UNIQUE
products	id, seller_id, category_id, name, description, price, stock, is_active, created_at, updated_at	FK seller_id, category_id; price > 0; stock >= 0; index on seller_id, category_id
product_images	id, product_id, image_url, is_primary, created_at	FK product_id (CASCADE)
carts	id, user_id, created_at, updated_at	user_id UNIQUE
cart_items	id, cart_id, product_id, quantity, created_at, updated_at	UNIQUE(cart_id, product_id); quantity > 0
addresses	id, user_id, full_name, phone, address_line, city, postal_code, country, is_default	FK user_id (CASCADE)
orders	id, buyer_id, shipping_address_id, total_amount, status, created_at, updated_at	FK buyer_id, shipping_address_id (RESTRICT)
seller_orders	id, order_id, seller_id, status, created_at, updated_at	FK order_id, seller_id; UNIQUE(order_id, seller_id)

Table	Columns	Key constraints
order_items	id, order_id, seller_order_id, product_id, product_name, quantity, unit_price, subtotal	FK order_id, seller_order_id, product_id (RESTRICT)
payments	id, order_id, amount, payment_method, status, paid_at, created_at	order_id UNIQUE; method = COD; status in (pending, completed)
reviews	id, user_id, product_id, rating, comment, created_at	UNIQUE(user_id, product_id); rating CHECK 1..5

5. Business Rules (yaad rakhne wali cheezein)

Seller order status transitions (Seller khud change karta hai)

Current	Allowed next	Kaun
pending	processing	Seller (apna order)
processing	shipped	Seller (apna order)
shipped	delivered	Seller (apna order)
pending / processing / shipped	cancelled	Sirf Admin
delivered / cancelled	koi change nahi	-

Main order status kaise nikalta hai (backend khud)

- Cancelled seller orders ko ignore karein. Agar tamam cancelled hon to order = **cancelled**.
- Baqi sab delivered hon to order = **delivered**.
- Baqi sab shipped ya delivered hon to order = **shipped**.
- Baqi mein se koi bhi processing/shipped/delivered ho to order = **processing**.
- Warna order = **pending**.

COD payment rules

- Checkout par payment banti hai: method = COD, status = pending, amount = order total, paid_at = NULL.
- Order delivered hone par payment **automatically completed nahi** hoti.
- Sirf Admin confirm-cod se payment completed hoti hai, aur sirf tab jab order status delivered ho.
- Confirm hone par backend khud paid_at = current UTC time set karta hai. Dobara confirm = 409.

Stock aur cancellation

- Checkout par stock kam hota hai (row lock ke saath). Order cancel hone par stock wapas add hota hai.
- Product delete = soft delete (is_active = false). Purane orders ka snapshot (name, price) change nahi hota.
- Delivery charge: config mein `DELIVERY_CHARGE` (default 0). Total = items total + delivery charge.

Review rule

- Sirf buyer, aur sirf us product par jo uske kisi **delivered** seller order mein ho. Ek buyer ek product par ek review. Rating 1 se 5.

6. Master API List: Kaun Si API Kis Din

Day	Method	Endpoint	Access	Purpose
Day 1	GET	`/health`	Public	App chal raha hai
Day 2	GET	`/health/db`	Public	Database connected hai
Day 3	POST	`/auth/register`	Public	Buyer/Seller signup
Day 3	POST	`/auth/login`	Public	Login, JWT milta hai
Day 6	GET	`/auth/me`	Any logged-in	Current user
Day 7	POST	`/categories/`	Admin	Category create
Day 7	GET	`/categories/`	Public	Categories list

Day	Method	Endpoint	Access	Purpose
Day 7	PUT	`/categories/{id}`	Admin	Category update
Day 7	DELETE	`/categories/{id}`	Admin	Category delete
Day 8	POST	`/products/`	Seller	Product create
Day 8	GET	`/products/`	Public	Active products + filters
Day 8	GET	`/products/{id}`	Public	Product details
Day 8	PUT	`/products/{id}`	Seller (own)	Product update
Day 8	DELETE	`/products/{id}`	Seller (own)	Product deactivate
Day 8	GET	`/admin/products`	Admin	Sab products (inactive bhi)
Day 9	POST	`/products/{id}/images`	Seller (own)	Multiple images upload
Day 9	DELETE	`/product-images/{id}`	Seller (own)	Image delete
Day 10	POST	`/addresses/`	Buyer	Address add
Day 10	GET	`/addresses/`	Buyer	Apne addresses
Day 10	PUT	`/addresses/{id}`	Buyer (own)	Address update
Day 10	DELETE	`/addresses/{id}`	Buyer (own)	Address delete
Day 10	GET	`/cart/`	Buyer	Cart dekhna
Day 10	POST	`/cart/items`	Buyer	Cart mein add
Day 10	PATCH	`/cart/items/{id}`	Buyer (own)	Quantity update
Day 10	DELETE	`/cart/items/{id}`	Buyer (own)	Item remove
Day 11	POST	`/orders/`	Buyer	Checkout
Day 12	GET	`/orders/my-orders`	Buyer	Order history
Day 12	GET	`/orders/{id}`	Buyer (own) / Admin	Order details
Day 12	GET	`/seller/orders`	Seller	Apne seller orders
Day 12	PATCH	`/seller/orders/{id}/status`	Seller (own)	Status update
Day 13	GET	`/payments/{id}`	Buyer (own) / Admin	Payment details
Day 13	PATCH	`/admin/payments/{id}/confirm-cod`	Admin	COD cash confirm
Day 13	GET	`/admin/orders`	Admin	Sab orders
Day 13	PATCH	`/admin/orders/{id}/cancel`	Admin	Order cancel + stock restore
Day 13	GET	`/admin/users`	Admin	Users list
Day 13	PATCH	`/admin/users/{id}/status`	Admin	User activate/deactivate
Day 14	POST	`/products/{id}/reviews`	Buyer (verified)	Review add
Day 14	GET	`/products/{id}/reviews`	Public	Reviews dekhna

Total: **38 endpoints** (Day 15 par koi naya API nahi; sirf testing aur documentation).

DAY 1: Project Setup aur Environment

Goal	Project ka base, virtual environment, dependencies aur pehla chalta hua FastAPI server.
Andaza waqt	2 - 3 ghante
Pehle kya zaroori	Python 3.11+, PostgreSQL, Git, VS Code install hon.
Aaj ki APIs	1 endpoint(s)
Migration	Koi nahi

Steps (is order mein karein)

1. Project folder banayein: mkdir ecommerce_backend, phir cd ecommerce_backend.
2. Git init: git init. Virtual env: python -m venv venv, phir activate karein (Windows: venv\Scripts\activate).
3. Dependencies install karein (neeche command), phir pip freeze > requirements.txt.
4. Folders banayein: app/core, app/models, app/schemas, app/routers, app/services, app/repositories, app/dependencies, tests, uploads, scripts. Har package folder mein khali __init__.py.
5. app/main.py mein FastAPI app aur GET /health likhein (neeche code).
6. .gitignore banayein: venv/, .env, __pycache__/, uploads/*, *.pyc.
7. Server chalayein: uvicorn app.main:app --reload. Browser mein /docs kholein.
8. README.md mein project ka naam aur setup ka skeleton likhein. Pehla commit karein.

```
pip install fastapi "uvicorn[standard]" sqlalchemy psycopg2-binary alembic \
    pydantic-settings "python-jose[cryptography]" "passlib[bcrypt]" \
    python-multipart email-validator pytest httpx

# app/main.py
from fastapi import FastAPI
app = FastAPI(title="Multi-Vendor E-Commerce API", version="1.0.0")

@app.get("/health", tags=["Health"])
def health():
    return {"status": "ok"}
```

Aaj ki files

- app/main.py
- requirements.txt
- .gitignore
- README.md (skeleton)
- Khali __init__.py files

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /health`	Public	-	`{"status":"ok"}`	-

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D1-1	Server start	uvicorn app.main:app --reload	Console mein 'Application startup complete', koi error nahi
D1-2	Health check	GET /health	200 + {"status": "ok"}
D1-3	Swagger docs	Browser: localhost:8000/docs	Swagger UI khulta hai, /health nazar aata hai
D1-4	OpenAPI json	Browser: localhost:8000/openapi.json	JSON milta hai
D1-5	Git ignore	git status	venv aur .env track nahi ho rahe

Postman folder: Abhi zaroorat nahi (Day 5 se).

End-of-Day Checklist

[ ] Virtual env active hai
[ ] requirements.txt bana hai
[ ] /health 200 deta hai
[ ] /docs khulta hai
[ ] Folder structure complete hai
[ ] Pehla commit ho gaya

Git commit: `chore: initial FastAPI project setup`

DAY 2: Configuration aur PostgreSQL Connection

Goal	Settings .env se load hon aur app PostgreSQL se connect ho.
Andaza waqt	2 ghante
Pehle kya zaroori	Day 1 complete, PostgreSQL chal raha ho.
Aaj ki APIs	1 endpoint(s)
Migration	Koi nahi. Yaad rakhein: create_all() nahi chalana.

Steps (is order mein karein)

1. PostgreSQL mein database banayein: CREATE DATABASE ecommerce_db; (pgAdmin ya psql se).
2. .env banayein aur .env.example (bina secrets ke) bhi banayein.
3. app/core/config.py: Settings class (pydantic-settings) jo .env parhe.
4. app/core/database.py: engine, SessionLocal, Base (DeclarativeBase) aur get_db() dependency.
5. main.py mein GET /health/db likhein jo SELECT 1 chala kar connection check kare.
6. Test karein, phir galat password daal kar error behavior dekhein, phir sahi karein.

```
# .env
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/ecommerce_db
SECRET_KEY=change-this-to-a-long-random-string
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
UPLOAD_DIR=uploads
MAX_IMAGE_SIZE_MB=5
MAX_IMAGES_PER_PRODUCT=5
DELIVERY_CHARGE=0
```

Aaj ki files

- app/core/config.py
- app/core/database.py
- .env
- .env.example
- app/main.py (health/db add)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /health/db`	Public	-	`{"database":"connected"}`	500 agar DB down

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D2-1	DB connected	GET /health/db	200 + {"database": "connected"}
D2-2	Galat password	.env mein password galat kar ke restart, GET /health/db	Error (500) aata hai, app crash nahi hoti. Phir sahi kar ke dobara 200
D2-3	Settings load	print(settings.DATABASE_URL) temporary	Value .env wali aati hai (test ke baad print hata dein)
D2-4	Secrets safe	git status	.env track nahi ho rahi, .env.example track ho rahi hai

Postman folder: Is din koi API folder nahi, sirf /health/db ko Swagger mein check karein.

End-of-Day Checklist

[ ] ecommerce_db bana hai

[ ] /health/db 200 deta hai

[ ] .env gitignore mein hai

[ ] .env.example commit hai

Git commit: `feat: add settings and PostgreSQL connection`

DAY 3: SQLAlchemy Models aur Relationships

Goal	Tamam 12 models, enums, constraints aur relationships likhna.
Andaza waqt	4 - 5 ghante
Pehle kya zaroori	Day 2 complete.
Aaj ki APIs	Koi naya API nahi
Migration	Aaj sirf models. Tables kal (Day 4) migration se banenge.

Steps (is order mein karein)

1. Enums banayein: UserRole (buyer, seller, admin), OrderStatus aur SellerOrderStatus (pending, processing, shipped, delivered, cancelled), PaymentMethod (COD), PaymentStatus (pending, completed).
2. Models is order mein likhein (dependency ki wajah se): user, category, address, product, product_image, cart, cart_item, order, seller_order, order_item, payment, review.
3. Har model mein PK, FK, nullable/default, created_at/updated_at (UTC) define karein.
4. Constraints lagayein: users.email unique, categories.name unique, carts.user_id unique, UniqueConstraint(cart_id, product_id), UniqueConstraint(user_id, product_id) reviews mein, payments.order_id unique, UniqueConstraint(order_id, seller_id) seller_orders mein, CheckConstraint(rating 1..5), CheckConstraint(stock >= 0), CheckConstraint(quantity > 0), CheckConstraint(price > 0).
5. Delete rules: product_images/cart_items/addresses = CASCADE; orders/order_items/seller_orders ke FK = RESTRICT (history safe).
6. Indexes: products(seller_id), products(category_id), orders(buyer_id), seller_orders(seller_id), order_items(order_id).
7. relationship() with back_populates likhein: User.products, Product.images, Cart.items, Order.seller_orders, SellerOrder.items, Order.payment, etc.
8. app/models/__init__.py mein sab models import karein (Alembic ko isi ki zaroorat hogi).

Aaj ki files

- app/models/user.py, category.py, address.py, product.py, product_image.py
- app/models/cart.py, cart_item.py, order.py, seller_order.py, order_item.py, payment.py, review.py
- app/models/enums.py
- app/models/__init__.py

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D3-1	Import check	python -c "from app.models import *"	Koi ImportError nahi
D3-2	Mapper check	python -c "from sqlalchemy.orm import configure_mappers; import app.models; configure_mappers()"	Koi relationship error nahi
D3-3	Model count	len(Base.metadata.tables)	12
D3-4	Relationships	Python shell: User, Product, Order, SellerOrder ke relationship attributes dekhein	user.products, product.images, order.seller_orders, seller_order.items, order.payment maujood hain
D3-5	Constraints	Base.metadata.tables['reviews'].constraints print karein	Unique(user_id, product_id) aur Check(rating) nazar aate hain

Postman folder: Aaj API nahi hai; Python shell mein check karein.

End-of-Day Checklist

[ ] 12 model files bani hain

[ ] Sab FKs aur relationships define hain

[ ] Constraints aur indexes lage hain

[ ] Import aur mapper check pass

Git commit: `feat: add all 12 SQLAlchemy models and relationships`

DAY 4: Alembic Setup aur Pehli Migration

Goal	Alembic configure kar ke 12 tables ko migration se create karna.
Andaza waqt	2 - 3 ghante
Pehle kya zaroori	Day 3 complete (models ready).
Aaj ki APIs	Koi naya API nahi
Migration	**Aaj pehli migration:** create all tables.

Steps (is order mein karein)

1. alembic init alembic chalayein.
2. alembic.ini mein sqlalchemy.url khali chorein (hum env.py se denge).
3. alembic/env.py mein: settings.DATABASE_URL set karein aur target_metadata = Base.metadata, aur app.models import karein.
4. Migration generate karein: alembic revision --autogenerate -m "create all tables".
5. Generated file khol kar review karein: 12 tables, enums, constraints, indexes, delete rules sahi hain?
6. Apply karein: alembic upgrade head.
7. pgAdmin/psql mein verify karein: \dt se 12 tables + alembic_version.
8. Downgrade/upgrade test karein, phir database ko head par wapas laayein.

```
alembic init alembic
alembic revision --autogenerate -m "create all tables"
alembic upgrade head
alembic current
alembic downgrade base      # test ke liye
alembic upgrade head        # wapas head par
```

Aaj ki files

- alembic.ini
- alembic/env.py
- alembic/versions/xxxx_create_all_tables.py

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D4-1	Tables bane	psql: \dt	12 tables + alembic_version
D4-2	Revision	alembic current	Latest revision ID dikhta hai (head)
D4-3	Downgrade	alembic downgrade base	Tamam tables delete (sirf alembic_version bacha)
D4-4	Upgrade dobara	alembic upgrade head	12 tables wapas aa gaye
D4-5	FK check	psql: \d products	FK to users aur categories, aur CHECK constraints nazar aate hain
D4-6	Unique check	psql: 2 users same email insert karein	Duplicate key error aata hai

Postman folder: Aaj API nahi; psql / pgAdmin se verify karein.

End-of-Day Checklist

[ ] Migration file review ho gayi

[ ] 12 tables database mein hain

[ ] upgrade/downgrade dono chalte hain

[ ] Migration Git mein commit hai

Git commit: `feat: add Alembic and initial migration (12 tables)`

DAY 5: Registration aur Login

Goal	Buyer/Seller signup, password hashing aur login se JWT token.
Andaza waqt	4 ghante
Pehle kya zaroori	Day 4 complete (users table maujood).
Aaj ki APIs	2 endpoint(s)
Migration	Aam tor par koi nahi (users table Day 4 mein ban chuki hai).

Steps (is order mein karein)

1. core/security.py: hash_password(), verify_password() (bcrypt), create_access_token() (agle din mukammal hoga, aaj basic).
2. schemas/user.py: UserCreate (name, email, password, role), UserResponse (id, name, email, role, is_active, created_at), LoginRequest, TokenResponse.
3. UserCreate mein role sirf Literal['buyer','seller']. Password rule: kam az kam 8 chars, upper, lower, digit, special.
4. repositories/user_repository.py: get_by_email(), get_by_id(), create().
5. services/auth_service.py: register (duplicate email = 409, hash, save) aur login (verify, is_active check, token).
6. routers/auth.py: POST /auth/register aur POST /auth/login. main.py mein router include karein.
7. Buyer ke liye cart: pehli baar cart use hone par automatically banega (get_or_create_cart), register par nahi.
8. Postman collection aur environment variables banayein. Test users (Ali, Sara, Ahmed, Bilal) register karein.
9. Login karein aur tokens Postman environment mein save karein.

Aaj ki files

- core/security.py
- schemas/user.py
- repositories/user_repository.py
- services/auth_service.py
- routers/auth.py

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /auth/register`	Public	`{"name":"Ali Khan","email":"ali@example.com","password":"Ali@12345","role":"buyer"}`	`201 {"id":1,"name":"Ali Khan","email":"ali@example.com","role":"buyer","is_active":true}`	409 email exists; 422 invalid role/email/password
`POST /auth/login`	Public	`{"email":"ali@example.com","password":"Ali@12345"}`	`200 {"access_token":"eyJ...","token_type":"bearer"}`	401 wrong credentials; 403 inactive user

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D5-1	Buyer register	POST /auth/register: Ali, role buyer	201, role = buyer, password_hash response mein nahi
D5-2	Seller register	POST /auth/register: Ahmed, role seller	201, role = seller
D5-3	Baqi test users	Sara (buyer), Bilal (seller) register karein	Dono 201
D5-4	Duplicate email	Ali ko dobara register karein	409 Conflict
D5-5	Admin role block	role = "admin" bhejein	422 (admin allowed nahi)
D5-6	Invalid email	email = "abc"	422
D5-7	Weak password	password = "12345"	422
D5-8	Missing field	name nahi bheja	422
D5-9	DB check	psql: SELECT email, password_hash FROM users	password_hash bcrypt ($2b$...) hai, plain text nahi
D5-10	Login sahi	POST /auth/login Ali	200 + access_token
D5-11	Login galat password	Ali + galat password	401
D5-12	Login unknown email	nobody@example.com	401

Postman folder: Day 5 - Auth (Register Buyer, Register Seller, Login Buyer, Login Seller, error cases).

End-of-Day Checklist

[ ] Buyer aur Seller register ho rahe hain

[ ] Admin role register nahi ho sakta

[ ] Password hashed hai

[ ] Login token deta hai

[ ] Postman environment mein 4 tokens save hain

Git commit: `feat: add user registration and login`

DAY 6: JWT Verification, Role-Based Access aur Admin Provisioning

Goal	Har protected API token verify kare, role check ho aur admin secure tareeqe se bane.
Andaza waqt	3 - 4 ghante
Pehle kya zaroori	Day 5 complete.
Aaj ki APIs	1 endpoint(s)
Migration	Koi nahi (agar admin ke liye extra column na chahiye).

Steps (is order mein karein)

1. create_access_token mein payload: sub = user id, role, exp (ACCESS_TOKEN_EXPIRE_MINUTES).
2. dependencies/auth.py: OAuth2PasswordBearer(tokenUrl='/auth/login'), get_current_user() (decode, user fetch, is_active check, warna 401).
3. Role guards: require_buyer, require_seller, require_admin (ya require_roles('seller','admin')). Galat role = 403.
4. GET /auth/me likhein.
5. scripts/create_admin.py: ADMIN_EMAIL/ADMIN_PASSWORD (env ya CLI args) se admin create kare. Public API se admin nahi banega.
6. Admin script chalayein: python -m scripts.create_admin. Admin login karein, admin_token Postman mein save karein.
7. Temporary test routes (debug) banayein: /_t/buyer, /_t/seller, /_t/admin jo guards check karein. Test ke baad hata dein.

Aaj ki files

- dependencies/auth.py
- core/security.py (update)
- routers/auth.py (/me add)
- scripts/create_admin.py

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /auth/me`	Any logged-in	Header: Bearer token	`200 {"id":1,"name":"Ali Khan","email":"ali@example.com","role":"buyer"}`	401 token missing/invalid/expired

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D6-1	Token ke baghair	GET /auth/me (no Authorization)	401
D6-2	Garbage token	Bearer abc123	401
D6-3	Expired token	ACCESS_TOKEN_EXPIRE_MINUTES=1 kar ke 1 min baad	401
D6-4	Valid buyer token	GET /auth/me with buyer_token	200, role = buyer
D6-5	Valid seller token	GET /auth/me with seller_token	200, role = seller
D6-6	Admin create	python -m scripts.create_admin, phir admin login	Login 200, /auth/me role = admin
D6-7	Buyer on seller route	GET /_t/seller with buyer_token	403
D6-8	Seller on admin route	GET /_t/admin with seller_token	403
D6-9	Admin on admin route	GET /_t/admin with admin_token	200
D6-10	Inactive user	psql: UPDATE users SET is_active=false WHERE email='sara@example.com'; phir Sara ka purana token	401/403, aur login 403. Test ke baad is_active true kar dein

Postman folder: Day 6 - Auth Guards (me, role tests).

End-of-Day Checklist

[ ] Token verify ho raha hai

[ ] Roles ke guards kaam kar rahe hain

[ ] Admin sirf script se banta hai

[ ] admin_token Postman mein hai

[ ] Temporary test routes hata di gayin

Git commit: `feat: add JWT auth, role guards and admin seed script`

DAY 7: Category APIs

Goal	Admin categories manage kare, sab categories dekh saken.
Andaza waqt	2 - 3 ghante
Pehle kya zaroori	Day 6 complete (admin_token ready).
Aaj ki APIs	4 endpoint(s)
Migration	Optional: default categories ko data migration ki jagah seed script se bhi daal sakte hain.

Steps (is order mein karein)

1. schemas/category.py: CategoryCreate (name, description), CategoryUpdate, CategoryResponse.
2. repositories/category_repository.py aur services/category_service.py: duplicate name check (409), not found (404).
3. Delete rule: agar category mein products hon to delete block (400, 'Category has products').
4. routers/categories.py: 4 endpoints, create/update/delete par require_admin.
5. Default categories add karein: Electronics, Clothing, Shoes, Mobiles, Accessories (Swagger se ya seed script se).
6. Sab tests chalayein.

Aaj ki files

- schemas/category.py
- repositories/category_repository.py
- services/category_service.py
- routers/categories.py

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /categories/`	Admin	`{"name":"Mobiles","description":"Smartphones"}`	`201 {"id":4,"name":"Mobiles","description":"Smartphones"}`	401, 403, 409 duplicate, 422
`GET /categories/`	Public	-	200 [{id, name, description}, ...]	-
`PUT /categories/{id}`	Admin	`{"name":"Mobile Phones"}`	200 updated category	403, 404, 409, 422
`DELETE /categories/{id}`	Admin	-	`200 {"message":"Category deleted"}`	403, 404, 400 has products

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D7-1	Admin create	POST /categories/ admin_token, name Mobiles	201
D7-2	5 categories	Electronics, Clothing, Shoes, Mobiles, Accessories create karein	Sab 201
D7-3	Duplicate	Mobiles dobara create	409
D7-4	Buyer create	POST /categories/ buyer_token	403
D7-5	Seller create	POST /categories/ seller_token	403
D7-6	No token	POST /categories/ without token	401
D7-7	Public list	GET /categories/ (no token)	200, 5 categories
D7-8	Update	PUT /categories/4 admin_token	200, name badal gaya
D7-9	Update not found	PUT /categories/9999	404
D7-10	Empty name	name = ""	422
D7-11	Delete	DELETE khali category, admin_token	200, phir GET list mein nahi

Postman folder: Day 7 - Categories.

End-of-Day Checklist

[ ] 4 category APIs kaam kar rahi hain

[ ] Sirf admin write kar sakta hai

[ ] 5 default categories maujood hain

[ ] Duplicate name block hai

Git commit: `feat: add category CRUD APIs`

DAY 8: Product CRUD

Goal	Seller apne products manage kare; buyers active products browse karein.
Andaza waqt	4 - 5 ghante
Pehle kya zaroori	Day 7 complete (categories maujood).
Aaj ki APIs	6 endpoint(s)
Migration	Optional: search/filter ke liye index (e.g. products.name) chahiye ho to nayi migration banayein.

Steps (is order mein karein)

1. schemas/product.py: ProductCreate (category_id, name, description, price > 0, stock >= 0), ProductUpdate (sab optional), ProductResponse (seller_id, category, is_active, images).
2. repositories/product_repository.py: create, get_by_id, list_active (filters + pagination), update, deactivate.
3. services/product_service.py: category exist check (400), ownership check (403), seller_id hamesha token se.
4. routers/products.py: POST (require_seller), GET list/detail (public, sirf active), PUT/DELETE (require_seller + owner).
5. GET /products/ filters: category_id, search (name ilike), min_price, max_price, seller_id, skip, limit.
6. DELETE = soft delete (is_active = false). Product detail mein inactive product public ko 404 de.
7. routers/admin.py: GET /admin/products (require_admin, inactive bhi, filters).
8. Test products banayein: Ahmed ke 2 (Samsung Mobile, Headphones), Bilal ka 1 (Nike Shoes).

Aaj ki files

- schemas/product.py
- repositories/product_repository.py
- services/product_service.py
- routers/products.py
- routers/admin.py

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /products/`	Seller	`{"category_id":4,"name":"Samsung Mobile","description":"...","price":50000,"stock":10}`	201 product (seller_id token se)	403 not seller; 400 bad category; 422
`GET /products/`	Public	Query: category_id, search, min_price, max_price, seller_id, skip, limit	200 list (sirf active)	422 bad query
`GET /products/{id}`	Public	-	200 product + images	404 not found/inactive
`PUT /products/{id}`	Seller own	`{"price":48000,"stock":12}`	200 updated product	403 not owner; 404; 422
`DELETE /products/{id}`	Seller own	-	200 product (is_active false)	403 not owner; 404
`GET /admin/products`	Admin	Query: filters, skip, limit	200 sab products	403

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D8-1	Seller create	POST /products/ seller_token (Samsung Mobile)	201, seller_id = Ahmed ka id
D8-2	Baqi products	Headphones (Ahmed), Nike Shoes (Bilal seller2_token)	Dono 201
D8-3	Buyer create	POST /products/ buyer_token	403
D8-4	Body mein seller_id	Body mein seller_id = 99 bhejein	Ignore hota hai; seller_id token wala hi rehta hai
D8-5	Bad category	category_id = 9999	400
D8-6	Negative price	price = -5	422
D8-7	Negative stock	stock = -1	422
D8-8	Public list	GET /products/ (no token)	200, 3 products
D8-9	Filter category	GET /products/?category_id=4	Sirf Mobiles
D8-10	Search	GET /products/?search=nike	Nike Shoes
D8-11	Price range	GET /products/?min_price=2000&max;_price=9000	Headphones aur Nike Shoes
D8-12	Pagination	GET /products/?limit=2&skip;=0	2 products
D8-13	Detail	GET /products/{id}	200 details
D8-14	Own update	Ahmed PUT apna product	200
D8-15	Other seller update	Bilal PUT Ahmed ka product	403
D8-16	Deactivate	Ahmed DELETE Headphones	200, is_active = false
D8-17	Hidden after delete	GET /products/ aur GET /products/{headphones_id}	List mein nahi, detail 404
D8-18	Admin view	GET /admin/products admin_token	200, inactive Headphones bhi nazar aata hai
D8-19	Reactivate	Ahmed PUT Headphones is_active = true (ya DB se)	Aage ke tests ke liye Headphones active hona chahiye

Postman folder: Day 8 - Products.

End-of-Day Checklist

[ ] 6 product APIs kaam kar rahi hain

[ ] Ownership checks pass

[ ] 3 test products maujood aur active hain

[ ] Soft delete verified

Git commit: `feat: add product CRUD with ownership and filters`

DAY 9: Multiple Product Image Upload

Goal	Seller product ki multiple images upload aur delete kar sake.
Andaza waqt	3 - 4 ghante
Pehle kya zaroori	Day 8 complete.
Aaj ki APIs	2 endpoint(s)
Migration	Koi nahi (product_images table pehle se maujood hai).

Steps (is order mein karein)

1. uploads/ folder banayein aur main.py mein StaticFiles mount karein: app.mount('/uploads', StaticFiles(directory='uploads'), name='uploads').
2. services/image_service.py: validate (type jpg/jpeg/png/webp, size <= MAX_IMAGE_SIZE_MB, product par max MAX_IMAGES_PER_PRODUCT).
3. File ka asal content (magic bytes) bhi check karein, sirf extension par bharosa na karein.
4. Filename uuid4 + extension se banayein (original filename use nahi hoga, path traversal se bachao).
5. File uploads/products/{product_id}/ mein save karein; DB mein sirf URL (e.g. /uploads/products/1/abc.jpg).
6. Pehli image automatically is_primary = true.
7. routers/product_images.py: POST /products/{id}/images (List[UploadFile]) aur DELETE /product-images/{id}.
8. Delete par DB row aur disk file dono hatayein; primary delete ho to agli image primary banao.
9. ProductResponse mein images list show ho (Day 8 ka schema update).

Aaj ki files

- services/image_service.py
- schemas/product_image.py
- routers/product_images.py
- main.py (static mount)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /products/{id}/images`	Seller own	multipart/form-data: files (1 ya zyada)	`201 [{"id":1,"image_url":"/uploads/products/1/a.jpg","is_primary":true}]`	403 not owner; 404; 400 invalid type/size/limit
`DELETE /product-images/{id}`	Seller own	-	`200 {"message":"Image deleted"}`	403 not owner; 404

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D9-1	Single upload	Ahmed POST /products/1/images, 1 jpg	201, is_primary = true
D9-2	Multiple upload	Ek request mein 2 aur images	201, 2 rows, is_primary = false
D9-3	URL khulta hai	Browser: localhost:8000 + image_url	Image display hoti hai
D9-4	DB mein URL	psql: SELECT image_url FROM product_images	Sirf path/URL, file data nahi
D9-5	Invalid type	.pdf ya .exe upload	400
D9-6	Fake extension	Text file ko .jpg rename kar ke upload	400 (content check)
D9-7	Oversize	6 MB image	400
D9-8	Limit	Product par 6th image upload	400
D9-9	Other seller upload	Bilal Ahmed ke product par upload	403
D9-10	Buyer upload	buyer_token	403
D9-11	Product detail	GET /products/1	images list mein sab images
D9-12	Delete image	Ahmed DELETE /product-images/{id}	200, file disk se bhi gayab
D9-13	Delete primary	Primary image delete karein	Agli image primary ban jati hai
D9-14	Other seller delete	Bilal Ahmed ki image delete kare	403

Postman folder: Day 9 - Product Images.

End-of-Day Checklist

[ ] Multi-upload kaam karta hai

[ ] Validations pass

[ ] Ownership verified

[ ] Files uploads/ mein hain, git mein nahi

Git commit: `feat: add multiple product image upload`

DAY 10: Buyer Addresses aur Cart

Goal	Buyer address manage kare aur cart mein products add/update/remove kare.
Andaza waqt	5 - 6 ghante
Pehle kya zaroori	Day 8 complete (products maujood).
Aaj ki APIs	8 endpoint(s)
Migration	Koi nahi (addresses, carts, cart_items tables ban chuki hain).

Steps (is order mein karein)

1. **Addresses:** schemas/address.py (AddressCreate, AddressUpdate, AddressResponse), repository, service, routers/addresses.py. Har query mein user_id filter.
2. Pehla address automatically is_default = true. Naya default set ho to baqi ke is_default false.
3. Doosre buyer ka address: 403 (ya 404). Default address delete ho to koi aur default ban jaye.
4. **Cart:** services/cart_service.py: get_or_create_cart(user_id).
5. POST /cart/items: product active ho, quantity >= 1, total quantity stock se zyada na ho. Same product dobara add = quantity jama.
6. PATCH /cart/items/{id}: quantity update (ownership + stock check). DELETE /cart/items/{id}: item remove.
7. GET /cart/: items (product name, unit_price abhi ki price, quantity, subtotal) aur cart total.
8. Sab endpoints require_buyer. Seller/Admin ko 403.

Aaj ki files

- schemas/address.py, repositories/address_repository.py, services/address_service.py, routers/addresses.py
- schemas/cart.py, repositories/cart_repository.py, services/cart_service.py, routers/cart.py

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /addresses/`	Buyer	`{"full_name":"Ali Khan","phone":"03001234567","address_line":"House 12 Street 5","city":"Lahore","postal_code":"54000","country":"Pakistan","is_default":true}`	201 address	403; 422
`GET /addresses/`	Buyer	-	200 apne addresses	403
`PUT /addresses/{id}`	Buyer own	Address fields	200 updated	403 not owner; 404
`DELETE /addresses/{id}`	Buyer own	-	200 message	403; 404; 400 agar kisi order mein use hua ho
`GET /cart/`	Buyer	-	`200 {"items":[{"id":1,"product_id":1,"name":"Samsung Mobile","unit_price":"50000.00","quantity":1,"subtotal":"50000.00"}],"total":"50000.00"}`	403 not buyer
`POST /cart/items`	Buyer	`{"product_id":1,"quantity":1}`	201 cart item	400 stock/inactive; 404 product; 422
`PATCH /cart/items/{id}`	Buyer own	`{"quantity":3}`	200 updated item	403; 404; 400 stock; 422 qty < 1
`DELETE /cart/items/{id}`	Buyer own	-	200 message	403; 404

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D10-1	Add address	Ali POST /addresses/	201, is_default = true
D10-2	Second address	Ali doosra address is_default = true	201, pehla ab default nahi
D10-3	List own	Ali GET /addresses/	Sirf Ali ke 2 addresses
D10-4	Other buyer's address	Sara GET/PUT Ali ka address id	403/404
D10-5	Seller address	Ahmed POST /addresses/	403
D10-6	Empty cart	Ali GET /cart/	200, items = [], total = 0
D10-7	Add to cart	Ali POST /cart/items Mobile x1	201
D10-8	Same product again	Mobile x1 dobara add	Quantity = 2 (duplicate row nahi)
D10-9	Stock se zyada	Mobile x11 (stock 10)	400
D10-10	Inactive product	Inactive product add karein	400/404
D10-11	Qty 0	quantity = 0	422
D10-12	Update qty	PATCH item quantity = 1	200, subtotal update
D10-13	Cart total	Mobile x1, Headphones x2, Shoes x1 add kar ke GET /cart/	total = 64000.00
D10-14	Other buyer's item	Sara Ali ke cart item par PATCH/DELETE	403
D10-15	Sara ka cart	Sara GET /cart/	Khali (Ali ka cart nahi dikhta)
D10-16	Seller cart	Ahmed GET /cart/	403
D10-17	Remove item	Ali DELETE cart item	200, cart se gayab
D10-18	Cart wapas set	Remove ke baad Day 11 ke liye cart dobara 64000 wala banayein	Total 64000.00

Postman folder: Day 10 - Addresses and Cart.

End-of-Day Checklist

[ ] 8 APIs (4 address + 4 cart) kaam kar rahi hain

[ ] Buyer isolation verified

[ ] Cart mein duplicate rows nahi bantin

[ ] Cart total 64000 set hai (kal ke liye)

Git commit: `feat: add buyer addresses and cart management`

DAY 11: Checkout aur Multi-Vendor Order Creation

Goal	Cart se order, seller-wise sub-orders, order items aur COD payment ek hi transaction mein.
Andaza waqt	6 - 7 ghante (sab se important din)
Pehle kya zaroori	Day 10 complete, Ali ke cart mein 64000 ka saman.
Aaj ki APIs	1 endpoint(s)
Migration	Optional: delivery charge ke liye orders mein alag column chahiye ho to nayi migration. Default plan mein zaroorat nahi.

Steps (is order mein karein)

1. schemas/order.py: CheckoutRequest (shipping_address_id), OrderResponse (id, status, total_amount, address, seller_orders[items, status, subtotal], payment).
2. repositories/order_repository.py: create order, seller order, order items, payment.
3. services/order_service.py: checkout() neeche diye algorithm ke mutabiq. Poora function ek DB transaction mein.
4. Stock lock: products ko id ascending order mein lock karein taake deadlock na ho.
5. routers/orders.py: POST /orders/ (require_buyer), response 201.
6. Rollback test ke liye temporary exception (e.g. payment banne ke baad raise) lagayein, test kar ke hata dein.
7. Race test: do requests ek saath ek aakhri stock item par chalayein (do Postman tabs ya script).

Checkout ka algorithm (transaction ke andar)

```
POST /orders/  body: { "shipping_address_id": 1 }

with transaction:
  1. user buyer hai (guard)                          -> warna 403
  2. cart + items load karo; items khali?            -> 400 "Cart is empty"
  3. address = (id, user_id) se; nahi mila?          -> 400 "Invalid address"
  4. products ko id order mein lock karo (SELECT ... FOR UPDATE)
  5. har item: product.is_active? stock >= qty?      -> 400 (naam ke saath message)
  6. items ko product.seller_id se group karo
  7. total = sum(price * qty) + DELIVERY_CHARGE       (price DB se, client se nahi)
  8. Order(buyer, address, total, status=pending)
  9. har seller ke liye SellerOrder(status=pending)
 10. har item ke liye OrderItem(name, unit_price snapshot, qty, subtotal)
 11. Payment(order, amount=total, COD, pending, paid_at=None)
 12. product.stock -= qty
 13. cart_items delete (cart row rahe)
commit; koi bhi exception -> rollback (cart intact)
```

Aaj ki files

- schemas/order.py
- repositories/order_repository.py
- services/order_service.py
- routers/orders.py

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /orders/`	Buyer	`{"shipping_address_id": 1}`	`201 {"id":1,"status":"pending","total_amount":"64000.00","seller_orders":[{"seller_id":3,"status":"pending","subtotal":"56000.00"},{"seller_id":4,"status":"pending","subtotal":"8000.00"}],"payment":{"method":"COD","status":"pending","amount":"64000.00"}}`	403 not buyer; 400 empty cart / bad address / stock / inactive; 401

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D11-1	Seller checkout	Ahmed POST /orders/	403
D11-2	Empty cart	Sara (khali cart) POST /orders/	400 'Cart is empty'
D11-3	Invalid address	Ali shipping_address_id = 9999	400
D11-4	Other buyer's address	Ali, Sara ka address id	400
D11-5	Insufficient stock	Ali cart mein Shoes x6 (stock 5), checkout	400, kuch change nahi hua
D11-6	Inactive product	Cart ka product deactivate kar ke checkout	400
D11-7	Success	Ali cart = Mobile 1, Headphones 2, Shoes 1; POST /orders/	201, total_amount = 64000.00, status pending
D11-8	Seller orders	psql: SELECT * FROM seller_orders	2 rows: Ahmed (56000 ke items), Bilal (8000)
D11-9	Order items	psql: SELECT * FROM order_items	3 rows, product_name aur unit_price snapshot saved
D11-10	Payment row	psql: SELECT * FROM payments	1 row: COD, pending, amount 64000, paid_at NULL
D11-11	Stock updated	GET /products/	Mobile 9, Headphones 18, Shoes 4
D11-12	Cart cleared	Ali GET /cart/	items = [] (cart row db mein maujood)
D11-13	Price snapshot	Ahmed Mobile ki price 45000 kar de; order items dekhein	Purane order mein unit_price abhi bhi 50000
D11-14	Client total ignore	Body mein total_amount = 1 bhejein	Ignore hota hai, total backend ka
D11-15	Rollback	Temporary exception laga kar checkout	500, lekin orders/seller_orders/payments mein koi row nahi, stock same, cart intact
D11-16	Race condition	Do buyers ek saath last 1 stock item checkout karein	Sirf ek 201, doosra 400; stock kabhi negative nahi

Postman folder: Day 11 - Checkout (success aur sab failure cases).

End-of-Day Checklist

[ ] Checkout transaction mein chalta hai

[ ] 2 seller orders bante hain

[ ] Stock sahi kam hota hai

[ ] Cart items delete hote hain

[ ] Rollback test pass

[ ] Temporary exception hata di gayi

Git commit: `feat: add multi-vendor checkout with transaction and COD payment`

DAY 12: Order Views, Seller Orders aur Delivery Workflow

Goal	Buyer order track kare; seller apne orders dekhe aur status badle; main order status khud update ho.
Andaza waqt	5 - 6 ghante
Pehle kya zaroori	Day 11 complete (Ali ka order #1 maujood).
Aaj ki APIs	4 endpoint(s)
Migration	Optional: status change ka waqt (e.g. delivered_at) chahiye ho to nayi migration.

Steps (is order mein karein)

1. GET /orders/my-orders: Ali ke apne orders (pagination, newest first).
2. GET /orders/{id}: Buyer (owner) ya Admin. Doosra buyer = 403. Seller = 403.
3. GET /seller/orders: seller_id = token user; sirf apne seller_orders aur unke items (optional filter status).
4. PATCH /seller/orders/{id}/status: owner check (403), transition rules (invalid = 400): pending to processing to shipped to delivered. Cancelled sirf admin.
5. services/order_status_service.py: recalculate_order_status(order): Section 5 ke rule se main order status update karein. Har seller status change ke baad call karein.
6. Order response mein har seller order ka status dikhayein.

Aaj ki files

- services/seller_order_service.py
- services/order_status_service.py
- routers/seller_orders.py
- routers/orders.py (extend)
- schemas/order.py (extend)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /orders/my-orders`	Buyer	Query: skip, limit	200 [order...]	403
`GET /orders/{id}`	Buyer own / Admin	-	200 order + seller_orders + items + payment	403; 404
`GET /seller/orders`	Seller	Query: status, skip, limit	200 sirf apne seller orders	403
`PATCH /seller/orders/{id}/status`	Seller own	`{"status":"processing"}`	200 seller order + order_status	403 not owner; 400 invalid transition; 404

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D12-1	My orders	Ali GET /orders/my-orders	200, order #1
D12-2	Order details	Ali GET /orders/1	200, 2 seller orders, 3 items, payment pending
D12-3	Other buyer	Sara GET /orders/1	403
D12-4	Seller on order detail	Ahmed GET /orders/1	403
D12-5	Admin order detail	Admin GET /orders/1	200
D12-6	Ahmed list	Ahmed GET /seller/orders	Sirf apna seller order (Mobile + Headphones)
D12-7	Bilal list	Bilal GET /seller/orders	Sirf apna (Shoes)
D12-8	Buyer on seller list	Ali GET /seller/orders	403
D12-9	Other seller update	Ahmed PATCH Bilal ka seller order	403
D12-10	Invalid skip	Ahmed pending se seedha delivered	400
D12-11	Ahmed processing	Ahmed PATCH processing	200; main order status = processing
D12-12	Bilal processing	Bilal PATCH processing	200; order = processing
D12-13	Ahmed shipped	Ahmed PATCH shipped	200; order abhi processing (Bilal processing hai)
D12-14	Bilal shipped	Bilal PATCH shipped	200; order = shipped
D12-15	Ahmed delivered	Ahmed PATCH delivered	200; order abhi shipped (Bilal baqi)
D12-16	Bilal delivered	Bilal PATCH delivered	200; order = delivered
D12-17	Payment check	psql: SELECT status, paid_at FROM payments	Abhi bhi pending, paid_at NULL
D12-18	Delivered ke baad change	Ahmed delivered order ko processing karein	400
D12-19	Seller cancel	Seller PATCH status = cancelled	400/403 (sirf admin)

Postman folder: Day 12 - Orders and Seller Orders.

End-of-Day Checklist

[ ] Buyer apne orders dekh sakta hai

[ ] Seller sirf apne orders dekhta hai

[ ] Transitions enforce hoti hain

[ ] Main order status sahi derive hota hai

[ ] Payment delivered par bhi pending rehti hai

Git commit: `feat: add order views, seller orders and delivery status workflow`

DAY 13: COD Payment aur Admin Controls

Goal	Admin COD cash confirm kare, orders aur users manage kare, cancel kar sake.
Andaza waqt	5 - 6 ghante
Pehle kya zaroori	Day 12 complete (order #1 delivered, payment pending). Ek aur naya order #2 test ke liye banayein.
Aaj ki APIs	6 endpoint(s)
Migration	Optional: seller approval chahiye ho to users.is_approved column (nayi migration).

Steps (is order mein karein)

1. GET /payments/{id}: Buyer (apne order ki), Admin. Seller/doosra buyer = 403.
2. PATCH /admin/payments/{id}/confirm-cod: require_admin; payment pending ho (warna 409); order delivered ho (warna 400); status = completed; paid_at = datetime.now(UTC) backend set kare.
3. GET /admin/orders: sab orders, filters (status, buyer_id, seller_id), pagination.
4. PATCH /admin/orders/{id}/cancel: delivered order cancel nahi (400); tamam seller orders cancelled; stock wapas add; order = cancelled. Payment pending hi rehti hai (confirm ab mumkin nahi).
5. GET /admin/users: users list (filter role, is_active, pagination).
6. PATCH /admin/users/{id}/status: is_active true/false. Admin khud ko deactivate nahi kar sakta (400).
7. Deactivated user ka login aur purana token dono reject (Day 6 ka logic).

Aaj ki files

- services/payment_service.py
- schemas/payment.py
- routers/payments.py
- routers/admin.py (orders, payments, users add)
- services/admin_service.py

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /payments/{id}`	Buyer own / Admin	-	`200 {"id":1,"order_id":1,"amount":"64000.00","payment_method":"COD","status":"pending","paid_at":null}`	403; 404
`PATCH /admin/payments/{id}/confirm-cod`	Admin	- (body nahi)	`200 {"status":"completed","paid_at":"2026-11-12T10:30:00Z"}`	403; 404; 400 order not delivered; 409 already completed
`GET /admin/orders`	Admin	Query: status, buyer_id, seller_id, skip, limit	200 sab orders	403
`PATCH /admin/orders/{id}/cancel`	Admin	-	200 cancelled order	403; 404; 400 delivered/already cancelled
`GET /admin/users`	Admin	Query: role, is_active, skip, limit	200 users (password_hash nahi)	403
`PATCH /admin/users/{id}/status`	Admin	`{"is_active":false}`	200 updated user	403; 404; 400 self-deactivate

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D13-1	Buyer payment view	Ali GET /payments/1	200, status pending
D13-2	Other buyer	Sara GET /payments/1	403
D13-3	Seller payment view	Ahmed GET /payments/1	403
D13-4	Confirm before delivery	Order #2 (abhi pending) ka payment confirm	400
D13-5	Buyer confirm	Ali PATCH confirm-cod	403
D13-6	Seller confirm	Ahmed PATCH confirm-cod	403
D13-7	Admin confirm	Admin PATCH /admin/payments/1/confirm-cod (order #1 delivered)	200, status completed
D13-8	paid_at	Response aur psql check	paid_at not null, current UTC time
D13-9	Double confirm	Dobara confirm	409
D13-10	Order status unchanged	GET /orders/1	status delivered (payment ne order status nahi badla)
D13-11	Admin orders list	Admin GET /admin/orders	Sab orders
D13-12	Filter orders	GET /admin/orders?status=pending	Sirf pending
D13-13	Buyer admin orders	Ali GET /admin/orders	403
D13-14	Cancel pending order	Order #2 par PATCH cancel	200, order & seller orders cancelled
D13-15	Stock restored	GET /products/	Order #2 ka stock wapas add ho gaya
D13-16	Cancel delivered	Order #1 cancel	400
D13-17	Confirm cancelled order	Order #2 payment confirm	400
D13-18	Users list	Admin GET /admin/users	Sab users, password_hash nahi
D13-19	Deactivate user	Admin PATCH Sara is_active false	200
D13-20	Deactivated login	Sara login	403
D13-21	Reactivate	Admin PATCH Sara is_active true	200, login phir chalta hai
D13-22	Self deactivate	Admin apni id deactivate kare	400

Postman folder: Day 13 - Payments and Admin.

End-of-Day Checklist

[ ] COD confirm sirf admin kar sakta hai

[ ] paid_at backend set karta hai

[ ] Delivered par payment auto-complete nahi hoti

[ ] Cancel par stock restore hota hai

[ ] Admin user management kaam karta hai

Git commit: `feat: add COD confirmation and admin controls`

DAY 14: Reviews

Goal	Verified buyers product par rating aur comment de saken.
Andaza waqt	3 - 4 ghante
Pehle kya zaroori	Day 12 complete (Ali ka delivered order maujood).
Aaj ki APIs	2 endpoint(s)
Migration	Koi nahi (reviews table aur constraints Day 4 mein ban chuke hain).

Steps (is order mein karein)

1. schemas/review.py: ReviewCreate (rating 1..5, comment), ReviewResponse (id, user name, rating, comment, created_at).
2. services/review_service.py: verified purchase check: buyer ke kisi seller_order mein delivered status ke saath wo product ho (order_items + seller_orders + orders.buyer_id join).
3. Ek buyer ek product par ek hi review: duplicate = 409.
4. routers/reviews.py: POST /products/{id}/reviews (require_buyer), GET /products/{id}/reviews (public, pagination).
5. GET /products/{id} (aur list) mein average_rating aur review_count shamil karein (aggregate query).
6. Product exist/active check (404).

Aaj ki files

- schemas/review.py
- repositories/review_repository.py
- services/review_service.py
- routers/reviews.py
- schemas/product.py (average_rating, review_count add)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /products/{id}/reviews`	Buyer (verified)	`{"rating":5,"comment":"Great phone"}`	201 review	403 not verified / not buyer; 404; 409 duplicate; 422 rating
`GET /products/{id}/reviews`	Public	Query: skip, limit	200 reviews list	404 product

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D14-1	Not purchased	Sara POST review Mobile par (khareeda nahi)	403
D14-2	Purchased, delivered	Ali POST review Mobile par, rating 5	201
D14-3	Not delivered	Naya order jo abhi delivered nahi, us product par review	403
D14-4	Duplicate	Ali dobara Mobile par review	409
D14-5	Rating 0	rating = 0	422
D14-6	Rating 6	rating = 6	422
D14-7	Seller review	Ahmed POST review	403
D14-8	No token	POST review without token	401
D14-9	Public list	GET /products/1/reviews	200, Ali ka review
D14-10	Average rating	Ali ke baad 2nd verified buyer rating 3 de	average_rating = 4.0, review_count = 2
D14-11	Product not found	POST /products/9999/reviews	404

Postman folder: Day 14 - Reviews.

End-of-Day Checklist

[ ] Verified purchase rule kaam karta hai

[ ] Duplicate review block hai

[ ] Average rating product mein aati hai

Git commit: `feat: add verified product reviews`

DAY 15: Testing, Security, Documentation aur GitHub

Goal	Poore project ko automated tests, security review, README aur GitHub push se final karna.
Andaza waqt	6 - 8 ghante
Pehle kya zaroori	Day 1 se Day 14 complete.
Aaj ki APIs	Koi naya API nahi
Migration	**Final verification:** khali database par alembic upgrade head, alembic current, alembic history. README mein migration commands likhein.

Steps (is order mein karein)

1. tests/ mein pytest setup: alag test database (ecommerce_test), conftest.py fixtures.
2. Automated tests likhein (neeche list) jo Day 5 se 14 ke manual test cases ko cover karein.
3. Fresh database par migration verify: naya khali DB bana kar alembic upgrade head, tables check.
4. Security review list complete karein (neeche).
5. Global exception handlers aur consistent error format {"detail": ...} check karein.
6. Swagger: tags, summaries, example payloads add karein. Postman collection export karein (postman_collection.json).
7. README likhein: features, tech stack, setup, .env variables, migration commands, admin creation, run, test, API list, roles.
8. End-to-end demo (Section 8) ek baar poora chalayein.
9. requirements.txt freeze, .env.example final check, git commit aur GitHub par push.

pytest ki files

File	Kya test hoga
tests/conftest.py	Test DB, client, fixtures: buyer/seller/admin tokens, sample products
tests/test_auth.py	Register, login, admin role block, me, token errors
tests/test_rbac.py	Har role ke liye allowed/forbidden endpoints
tests/test_categories.py / test_products.py	CRUD, ownership, filters, soft delete
tests/test_images.py	Valid/invalid upload, limits, ownership
tests/test_cart_address.py	Isolation, stock limit, duplicate rows
tests/test_checkout.py	Success, failures, rollback, race condition
tests/test_seller_orders.py	Transitions, main status derivation
tests/test_payments.py	COD confirm rules, paid_at, auto-complete nahi hota
tests/test_reviews.py	Verified buyer, duplicate, rating

Security Review List

[ ] SECRET_KEY .env mein hai, repo mein nahi
[ ] Password hash bcrypt; response mein password_hash kabhi nahi
[ ] Har protected route par guard
[ ] Ownership checks (seller, buyer) har jagah
[ ] SQL injection: sirf ORM / parameterized queries
[ ] Image upload: type, size, content, uuid filename
[ ] CORS config sahi
[ ] Public signup se admin nahi
[ ] Error responses mein internal details leak nahi
[ ] Rate limiting login par (optional, recommended)

Aaj ki files

- tests/conftest.py aur tests/test_*.py
- README.md (complete)
- postman_collection.json
- .env.example (final)
- requirements.txt (final)

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D15-1	Test suite	pytest -v	Sab tests pass
D15-2	Fresh DB migration	Naya DB + alembic upgrade head	12 tables ban jate hain, koi error nahi
D15-3	Fresh clone	Naye folder mein clone, README ke steps follow	Project chal jata hai
D15-4	Secrets	git log / grep SECRET_KEY	Koi secret commit nahi hua
D15-5	E2E demo	Section 8 ke steps	Sab steps expected result dete hain
D15-6	15 scenarios	Section 8 ki list	Sab pass

Postman folder: Postman collection export karke repo mein rakhein.

End-of-Day Checklist

[ ] pytest sab pass
[ ] Migrations fresh DB par chalti hain
[ ] README complete
[ ] Postman collection export
[ ] Security list complete
[ ] GitHub par push ho gaya

Git commit: `docs: add tests, README and final documentation`

8. Final Checklist aur End-to-End Demo

15 Zaroori Testing Scenarios

#	Scenario	Din
1	Buyer product create na kar sake	Day 8
2	Seller checkout na kar sake	Day 11
3	Seller doosre Seller ka product update na kar sake	Day 8
4	Buyer doosre Buyer ka cart access na kar sake	Day 10
5	Empty cart se checkout fail ho	Day 11
6	Insufficient stock par checkout reject ho	Day 11
7	Invalid address par checkout reject ho	Day 11
8	Successful checkout par order create ho	Day 11
9	Successful checkout ke baad cart items clear hon	Day 11
10	Database error par transaction rollback ho	Day 11
11	Seller doosre Seller ka order update na kar sake	Day 12
12	Delivered order par payment automatically completed na ho	Day 12, 13
13	COD cash confirmation ke baad paid_at set ho	Day 13
14	Invalid image upload reject ho	Day 9
15	Public signup se admin account create na ho	Day 5, 6

End-to-End Demo (poora flow ek baar)

1. Admin script se admin banayein, admin login.
2. Admin 5 categories banaye.
3. Ahmed (seller) aur Bilal (seller) register + login.
4. Ahmed: Samsung Mobile aur Headphones create, images upload.
5. Bilal: Nike Shoes create.
6. Ali (buyer) register + login.
7. Ali GET /products/ se products dekhe.
8. Ali cart mein Mobile x1, Headphones x2, Shoes x1 add kare (total 64000).
9. Ali address add kare.
10. Ali POST /orders/ checkout: 201, 2 seller orders, payment COD pending.
11. Ali cart khali; stock kam.
12. Ahmed apne seller order ko processing, shipped, delivered kare.
13. Bilal apne seller order ko processing, shipped, delivered kare.
14. Ali GET /orders/{id}: status delivered, payment pending.
15. Admin confirm-cod kare: payment completed, paid_at set.
16. Ali Mobile par review de (verified).
17. Admin GET /admin/orders aur GET /admin/users check kare.

Final Project Checklist (24 Tasks)

#	Task	Din	Done
1	FastAPI project setup	Day 1	[ ]
2	PostgreSQL connection	Day 2	[ ]
3	12 database tables	Day 3	[ ]
4	Primary and foreign key relationships	Day 3	[ ]
5	Alembic migrations	Day 4	[ ]

#	Task	Din	Done
6	Separate Buyer and Seller registration	Day 5	[ ]
7	JWT authentication	Day 6	[ ]
8	Role-based permissions	Day 6	[ ]
9	Admin management	Day 13	[ ]
10	Product CRUD	Day 8	[ ]
11	Multiple product image uploads	Day 9	[ ]
12	Categories	Day 7	[ ]
13	Buyer addresses	Day 10	[ ]
14	Cart management	Day 10	[ ]
15	Multi-vendor checkout	Day 11	[ ]
16	Seller-specific orders	Day 12	[ ]
17	COD payment system	Day 13	[ ]
18	Cart clearing after successful checkout	Day 11	[ ]
19	Seller delivery status updates	Day 12	[ ]
20	Admin order management	Day 13	[ ]
21	Reviews	Day 14	[ ]
22	API testing	Day 15	[ ]
23	README and documentation	Day 15	[ ]
24	Git commits and GitHub push	Day 15	[ ]

Final Project Definition

Hum ek backend-only Multi-Vendor E-Commerce Marketplace develop karenge using FastAPI, PostgreSQL aur SQLAlchemy. Buyer products purchase karega, Seller apne products aur orders manage karega, aur Admin overall system manage karega. Multi-seller order mein backend seller-wise sub-orders banata hai. Payment sirf COD hai; cash collection confirm hone par payment completed hoti hai aur paid_at backend set karta hai. Checkout ke baad purchased cart items delete hote hain, lekin order history hamesha database mein rehti hai. Database schema Alembic migrations se manage hota hai.